

# NUCLIO DIGITAL SCHOOL

Máster en Data Science & Artificial Intelligence

---

TRABAJO DE FIN DE MÁSTER

---

## DSMarket: Sistema de Predicción de Demanda y Optimización del Abastecimiento mediante Machine Learning

---

Autores: Gabriela Alberico, Jorge Silva,  
Robert Tunzi, Matías Lannes,  
Alexis Labrador

Grupo: 2 — DSCESP 0925

Tutor: Matías José Hermida  
*Senior Data Scientist, CaixaBank BI*

Período: Septiembre 2025 – Abril 2026

Defensa: Abril de 2026

---

# Resumen Ejecutivo

**DSMarket** es una cadena de supermercados que opera 10 tiendas en tres ciudades de Estados Unidos (Nueva York, Boston y Filadelfia) con un catálogo de más de 3.000 productos. La ineficiencia en la gestión manual del inventario genera pérdidas por *stockouts* (ventas perdidas) y por exceso de stock (capital inmovilizado), motivando el desarrollo de este proyecto.

El presente Trabajo de Fin de Máster desarrolla un **sistema *end-to-end* de predicción de demanda y optimización del abastecimiento** para DSMarket, aplicando técnicas de Machine Learning sobre datos históricos de ventas del período 2011–2016 (~58,3 millones de registros a granularidad ítem-tienda-día).

La metodología comprende cinco fases diferenciadas:

1. **Análisis exploratorio (EDA) y limpieza**, con seis hallazgos accionables sobre patrones geográficos, semanales y la elasticidad precio-demanda.
2. **Feature Engineering**, con 28 variables construidas a partir de lags temporales, estadísticos de ventana deslizante y features de precio.
3. **Clustering** de productos ( $k=5$ ) y tiendas ( $k=3$ ) mediante K-Means, identificando segmentos con comportamientos de demanda diferenciados.
4. **Modelización predictiva**, comparando Baseline (media móvil 28 días), Ridge, Random Forest, CatBoost, LightGBM y **XGBoost**, con optimización bayesiana de hiperparámetros mediante Optuna.
5. **Sistema de abastecimiento automático**, que genera órdenes de reposición semanales con clasificación de prioridad (ALTA/NORMAL).

El modelo final (**XGBoost** post-Optuna) obtiene un **RMSE = 1,8994** en el conjunto de validación, representando una **mejora del 13,1%** sobre el baseline (RMSE = 2,18), superando el objetivo mínimo del 5% establecido en los objetivos del proyecto.

**Palabras clave:** predicción de demanda, retail forecasting, XGBoost, Gradient Boosting, Optuna, K-Means clustering, sistema de abastecimiento, MLOps, series temporales, M5 Forecasting.

---

# Índice

|                                                                          |          |
|--------------------------------------------------------------------------|----------|
| <b>Resumen Ejecutivo</b>                                                 | <b>1</b> |
| <b>Índice de tablas</b>                                                  | <b>3</b> |
| <b>1 Introducción</b>                                                    | <b>4</b> |
| 1.1 Contexto del proyecto . . . . .                                      | 4        |
| 1.2 Motivación . . . . .                                                 | 4        |
| 1.3 Estructura del documento . . . . .                                   | 5        |
| <b>2 Objetivos</b>                                                       | <b>6</b> |
| 2.1 Objetivo general . . . . .                                           | 6        |
| 2.2 Objetivos específicos . . . . .                                      | 6        |
| 2.3 Finalidad del proyecto . . . . .                                     | 7        |
| <b>3 Metodología Empleada</b>                                            | <b>8</b> |
| 3.1 Descripción del dataset . . . . .                                    | 8        |
| 3.2 Análisis Exploratorio de Datos (EDA) y Limpieza . . . . .            | 9        |
| 3.2.1 Limpieza de datos . . . . .                                        | 9        |
| 3.2.2 Hallazgos del EDA . . . . .                                        | 9        |
| 3.3 Feature Engineering . . . . .                                        | 10       |
| 3.3.1 Features de lag (memoria temporal) . . . . .                       | 11       |
| 3.3.2 Features temporales . . . . .                                      | 11       |
| 3.3.3 Features de precio . . . . .                                       | 12       |
| 3.3.4 Features de producto y tienda (aggregate features) . . . . .       | 12       |
| 3.3.5 Split temporal . . . . .                                           | 12       |
| 3.4 Clustering . . . . .                                                 | 13       |
| 3.4.1 Clustering de productos (k = 5) . . . . .                          | 13       |
| 3.4.2 Clustering de tiendas (k = 3) . . . . .                            | 13       |
| 3.5 Modelos de predicción . . . . .                                      | 13       |
| 3.5.1 Justificación de la elección de Gradient Boosting con lag features | 13       |
| 3.5.2 Diferencias entre implementaciones de Gradient Boosting . . .      | 15       |
| 3.6 Optimización de hiperparámetros (Optuna) . . . . .                   | 15       |
| 3.7 Sistema de abastecimiento . . . . .                                  | 16       |
| 3.7.1 Lógica de cálculo de la orden . . . . .                            | 16       |
| 3.7.2 Clasificación por rotación de inventario . . . . .                 | 17       |
| 3.8 MLOps — Propuesta de puesta en producción . . . . .                  | 17       |

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| 3.8.1    | Paso 0 — Refactorización del código . . . . .           | 17        |
| 3.8.2    | Paso 1 — Empaquetado del modelo . . . . .               | 18        |
| 3.8.3    | Paso 2 — Pipeline de predicción semanal . . . . .       | 18        |
| 3.8.4    | Paso 3 — Monitoreo del modelo en producción . . . . .   | 18        |
| 3.8.5    | Paso 4 — Reentrenamiento . . . . .                      | 19        |
| 3.8.6    | Paso 5 — Versionado y trazabilidad . . . . .            | 19        |
| 3.8.7    | Diagrama del ciclo MLOps . . . . .                      | 20        |
| <b>4</b> | <b>Resultados y Conclusiones</b>                        | <b>21</b> |
| 4.1      | Resultados del EDA . . . . .                            | 21        |
| 4.2      | Resultados del Clustering . . . . .                     | 21        |
| 4.2.1    | Clustering de productos (k = 5) . . . . .               | 21        |
| 4.2.2    | Clustering de tiendas (k = 3) . . . . .                 | 21        |
| 4.3      | Comparativa de modelos predictivos . . . . .            | 22        |
| 4.3.1    | Nota sobre el MAPE en validación (55,82%) . . . . .     | 22        |
| 4.4      | Importancia de variables (Feature Importance) . . . . . | 24        |
| 4.5      | Sistema de reposición de inventario . . . . .           | 25        |
| 4.6      | Conclusiones finales . . . . .                          | 25        |
| 4.6.1    | Conclusiones técnicas . . . . .                         | 25        |
| 4.6.2    | Conclusiones de negocio . . . . .                       | 25        |
| 4.7      | Trabajo futuro . . . . .                                | 26        |
| <b>5</b> | <b>Referencias Bibliográficas</b>                       | <b>27</b> |
|          | Algoritmos y modelos . . . . .                          | 27        |
|          | Librerías de Python . . . . .                           | 27        |
|          | Dataset y competición de referencia . . . . .           | 27        |
|          | Metodología . . . . .                                   | 28        |
| <b>A</b> | <b>ML Canvas — DSMarket</b>                             | <b>29</b> |

---

# Índice de cuadros

|      |                                                                            |    |
|------|----------------------------------------------------------------------------|----|
| 2.1  | Objetivos específicos del proyecto DSMarket . . . . .                      | 6  |
| 3.1  | Fuentes de datos utilizadas en el proyecto . . . . .                       | 8  |
| 3.2  | Features de lag y estadísticos de ventana deslizante . . . . .             | 11 |
| 3.3  | Features temporales derivadas del calendario . . . . .                     | 11 |
| 3.4  | Features de precio y sus variaciones . . . . .                             | 12 |
| 3.5  | Aggregate features de producto y tienda . . . . .                          | 12 |
| 3.6  | División temporal del dataset . . . . .                                    | 12 |
| 3.7  | Perfil de los 5 clústeres de productos . . . . .                           | 14 |
| 3.8  | Perfil de los 3 clústeres de tiendas . . . . .                             | 14 |
| 3.9  | Comparativa de modelos — rendimiento en test . . . . .                     | 15 |
| 3.10 | Espacio de búsqueda de hiperparámetros XGBoost en Optuna . . . . .         | 16 |
| 3.11 | Artefactos del modelo generados en el <i>notebook</i> . . . . .            | 18 |
| 3.12 | Métricas de monitoreo en producción . . . . .                              | 18 |
| 3.13 | Modalidades de reentrenamiento del modelo . . . . .                        | 19 |
| 4.1  | Resultados comparativos de todos los modelos . . . . .                     | 22 |
| 4.2  | Resumen de métricas del modelo final . . . . .                             | 23 |
| 4.3  | Top 20 variables por importancia en el modelo XGBoost optimizado . . . . . | 24 |
| 4.4  | Ejemplo de output del sistema de reposición . . . . .                      | 25 |
| 4.5  | Áreas de mejora propuestas para iteraciones futuras . . . . .              | 26 |

# Introducción

## 1.1 Contexto del proyecto

El presente Trabajo de Fin de Máster se desarrolla en el marco del programa **Máster en Data Science & AI** de **Nuclio Digital School**, bajo la modalidad de proyecto Tipo 1 (*Role-play*) orientado al sector *Retail*. El escenario propuesto es el de **DSMarket**, una cadena de supermercados con presencia en tres ciudades de Estados Unidos (Nueva York, Boston y Filadelfia) que opera 10 tiendas y gestiona un catálogo de más de 3.000 productos.

La dirección de la compañía, representada por sus ejecutivos de nivel C (Paul, CFO, y Michelle, CDO), ha solicitado al equipo de Data Science el desarrollo de un sistema de inteligencia que permita **anticipar la demanda futura** de cada producto en cada tienda. El problema central es la ineficiencia en la gestión del inventario: DSMarket incurre en pérdidas por *stockouts* (productos agotados que generan ventas perdidas) y por **exceso de stock** (capital inmovilizado en productos que no se venden con la velocidad esperada).

La gestión manual del inventario, basada en criterios subjetivos o en medias históricas simples, no es capaz de capturar la complejidad de los patrones de demanda, que varían según el día de la semana, el mes del año, los eventos especiales (SuperBowl, Thanksgiving, New Year, etc.) y las fluctuaciones de precio.

## 1.2 Motivación

El presente proyecto aplica técnicas de **Machine Learning supervisado** sobre datos históricos de ventas para construir un modelo predictivo que supere al baseline y, a partir de sus predicciones, genere automáticamente **órdenes de reposición** para cada tienda. Este sistema representa el paso de una gestión *reactiva* (reponer cuando el stock es bajo) a una gestión **proactiva y basada en datos** (reponer antes de que ocurra el *stockout*).

**Motivación del proyecto**

El sector *retail* enfrenta un desafío estructural: los modelos tradicionales de gestión de inventario no capturan la heterogeneidad de la demanda a nivel de producto-tienda-día. Un modelo de Machine Learning entrenado sobre 58 millones de registros históricos puede capturar esas interacciones complejas y traducirlas en órdenes de reposición concretas, reduciendo tanto el riesgo de *stockout* como el coste del exceso de inventario.

### 1.3 Estructura del documento

Este documento se organiza en cuatro grandes bloques:

1. **Objetivos:** qué se pretende lograr con el proyecto.
2. **Metodología:** descripción de los datos, las técnicas aplicadas y las decisiones metodológicas tomadas.
3. **Resultados y Conclusiones:** presentación de los resultados obtenidos, las variables más influyentes y las conclusiones derivadas del análisis.
4. **Referencias bibliográficas:** referencias a los trabajos y herramientas utilizados.
5. **Anexo — ML Canvas:** descomposición estructurada del sistema mediante el marco *Machine Learning Canvas*.

# Objetivos

## 2.1 Objetivo general

Desarrollar un sistema *end-to-end* de **predicción de demanda y optimización del abastecimiento** para DSMarket, que permita a la dirección tomar decisiones informadas sobre la gestión del inventario, reduciendo los costes operativos y mejorando el nivel de servicio al cliente.

## 2.2 Objetivos específicos

La Tabla 2.1 resume los seis objetivos específicos del proyecto, junto con sus indicadores de éxito.

Cuadro 2.1: Objetivos específicos del proyecto DSMarket

| #  | Objetivo                                                                                                          | Indicador de éxito                                                                                                  |
|----|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| O1 | Realizar un análisis exploratorio completo (EDA) que identifique patrones de demanda, estacionalidad y anomalías  | Al menos 5 hallazgos de negocio accionables documentados y visualizados                                             |
| O2 | Segmentar productos y tiendas mediante clustering para identificar grupos con comportamientos diferenciados       | <i>Silhouette score</i> maximizado para cada nivel de granularidad, con interpretación de negocio clara por clúster |
| O3 | Construir un modelo de predicción de ventas que supere estadísticamente al baseline (media móvil 28 días) en RMSE | Mejora mínima del 5% en RMSE sobre el baseline en el conjunto de test                                               |



(continuación de la Tabla 2.1)

| #  | Objetivo                                                                                                | Indicador de éxito                                                                                                                     |
|----|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| O4 | Optimizar los hiperparámetros del modelo ganador mediante búsqueda bayesiana (Optuna)                   | RMSE en validación $\leq$ RMSE del modelo inicial sin optimizar                                                                        |
| O5 | Desarrollar un sistema automatizado de reposición de inventario que consuma las predicciones del modelo | Sistema capaz de generar órdenes de reposición para cualquier combinación tienda-producto con clasificación de prioridad (ALTA/NORMAL) |
| O6 | Exportar las predicciones y métricas a Power BI para su visualización y presentación ejecutiva          | Dashboard funcional con al menos 4 vistas diferenciadas                                                                                |

## 2.3 Finalidad del proyecto

La finalidad última de este proyecto es demostrar que las técnicas de Data Science pueden **crear valor de negocio tangible** en el sector *Retail*. La combinación de análisis exploratorio, clustering, predicción y sistema de abastecimiento proporciona a DS-Market una solución integral que no solo predice la demanda, sino que **traduce esas predicciones en acciones concretas**: las órdenes de reposición.

Adicionalmente, el proyecto sirve como demostración práctica de los conocimientos adquiridos a lo largo del Máster en Data Science, cubriendo el ciclo completo de un proyecto de ML: desde la obtención y limpieza de datos hasta la toma de decisiones basada en el modelo, incluyendo una **propuesta detallada de puesta en producción** (sección MLOps) que describe los pasos de despliegue, monitoreo y reentrenamiento que se seguirían en un entorno real.

## Metodología Empleada

### 3.1 Descripción del dataset

El proyecto parte de tres fuentes de datos internas de DSMarket, correspondientes al período **29 de enero de 2011 – 24 de abril de 2016** (1.913 días).

Cuadro 3.1: Fuentes de datos utilizadas en el proyecto

| Dataset                          | Descripción                                               | Dimensiones                       |
|----------------------------------|-----------------------------------------------------------|-----------------------------------|
| <code>item_sales.csv</code>      | Ventas diarias por producto-tienda en formato <i>wide</i> | 30.490 filas × 1.920 columnas     |
| <code>daily_calendar_with</code> | Calendario con fechas y eventos especiales                | 1.913 filas × 5 columnas          |
| <code>item_prices.csv</code>     | Precios semanales por producto-tienda                     | ~7 millones de filas × 5 columnas |

El dataset de ventas original tiene formato *wide*: cada fila es una combinación producto-tienda y cada columna representa un día (`d_1`, `d_2`, ..., `d_1913`). Mediante una operación de *unpivot* (transformación *wide*→*long* con `melt`), se convierte en un formato donde cada fila representa exactamente un registro (*item*, *store*, día)→ventas. Esto se denomina **granularidad ítem-tienda-día** e implica que cada fila responde a la pregunta: “¿cuántas unidades vendió este producto concreto, en esta tienda concreta, en este día concreto?”. Tras la transformación, el dataset consolidado contiene aproximadamente **58,3 millones de registros**.

#### Características del ámbito de análisis

- **3.049 productos únicos** organizados en 3 categorías (SUPERMARKET, ACCESSORIES, HOME\_&\_GARDEN) y 7 departamentos.
- **10 tiendas** en 3 ciudades: New York, Boston y Philadelphia.
- **Variable objetivo:** `sales` — ventas diarias en unidades, con distribución asimétrica positiva y cola larga.

## 3.2 Análisis Exploratorio de Datos (EDA) y Limpieza

### 3.2.1 Limpieza de datos

El proceso de limpieza se aplicó de forma diferenciada a cada fuente de datos.

#### Sales data

No se encontraron valores nulos ni duplicados. Los valores 0 en las columnas de ventas son datos reales: representan días en que un producto no se vendió, no registros faltantes. Se aplicó limpieza de espacios en los identificadores de texto.

#### Calendar data

La columna `event` contenía 1.887 *strings* vacíos (días sin evento especial). Estos se convirtieron correctamente a `NaN` en lugar de dejarse como *strings* vacíos, ya que la ausencia de evento es información, no un dato faltante. Adicionalmente se crearon *features* temporales derivadas: `year`, `month`, `day`, `quarter`, `week_of_year`, `is_weekend` e `is_event`.

#### Prices data

Se detectaron 243.920 valores nulos (3,50%) en la columna `yearweek`, distribuidos uniformemente entre todos los productos y tiendas. Se imputaron mediante la **mediana por grupo** (`item + store_code`), preservando el patrón de precios específico de cada producto en cada tienda. Se verificó que no existen precios negativos ni cero. Se confirmó la consistencia del catálogo: los 3.049 productos de ventas tienen correspondencia exacta en la tabla de precios. No se encontraron duplicados en ninguna de las tres fuentes.

### 3.2.2 Hallazgos del EDA

#### Hallazgo 1 — Liderazgo geográfico de New York

New York concentra el mayor volumen de ventas de las tres ciudades. Boston y Filadelfia presentan cifras muy similares entre sí. **Implicación de negocio:** las políticas de stock y reposición deben priorizarse por ciudad, no tratarse de forma uniforme.

#### Hallazgo 2 — Efecto fin de semana consistente y cuantificado

El sábado y el domingo generan entre un **27–33% más de ventas** que los días laborales en todas las ciudades. El miércoles es el día de menor actividad (“punto muerto” semanal). **Implicación:** los pedidos de reposición deben planificarse para que el stock llegue antes del jueves.

**Hallazgo 3 — SUPERMARKET es el motor del negocio**

La categoría SUPERMARKET representa el **68,7% de las ventas totales**. HOME & GARDEN aporta el 22% y ACCESSORIES el 9,3%. Dentro de SUPERMARKET, el departamento SUPERMARKET\_3 domina de forma clara. **Implicación:** cualquier fallo de stock en SUPERMARKET tiene un impacto desproporcionado en la facturación global.

**Hallazgo 4 — El pico de 2013 fue provocado por bajada de precios**

El análisis conjunto de ventas y precios revela que el gran pico histórico (septiembre 2013) coincide exactamente con una caída drástica de precios en los productos líderes. **Implicación:** la elasticidad precio-demanda es un *driver* real en este negocio, y las variables de precio son *features* relevantes para el modelo.

**Hallazgo 5 — Impacto diferenciado de los eventos especiales**

SuperBowl (+15,36% sobre la media) y Easter (+12,38%) disparan las ventas. Thanksgiving las reduce drásticamente (−40,21%), posiblemente por cierre de tiendas. **Implicación:** el calendario de eventos es una variable predictiva de alto valor y debe incorporarse al modelo.

**Hallazgo 6 — Fuerte autocorrelación en lag 1 ( $ACF \approx 0,78$ )**

Las ventas de ayer son el predictor individual más potente de las ventas de hoy. El test Augmented Dickey-Fuller confirma que la serie es **estacionaria** ( $p$ -value = 0,0008), por lo que no se requieren transformaciones adicionales (diferenciación, logaritmos). **Implicación metodológica:** los modelos de Gradient Boosting con *lag features* son la aproximación correcta para este problema.

### 3.3 Feature Engineering

Se construyó un **pipeline de feature engineering** (`feature_engineering_pipeline`) utilizando `sklearn.pipeline.Pipeline` y `sklearn.preprocessing.FunctionTransformer`. Las variables generadas se agrupan en cuatro familias.

### 3.3.1 Features de lag (memoria temporal)

**Cuadro 3.2:** Features de lag y estadísticos de ventana deslizante

| Feature                      | Descripción                                        |
|------------------------------|----------------------------------------------------|
| <code>sales_lag_1</code>     | Ventas del día anterior                            |
| <code>sales_lag_7</code>     | Ventas hace 7 días (mismo día de la semana pasada) |
| <code>sales_lag_14</code>    | Ventas hace 14 días                                |
| <code>sales_lag_21</code>    | Ventas hace 21 días                                |
| <code>sales_lag_28</code>    | Ventas hace 28 días                                |
| <code>rolling_mean_7</code>  | Media móvil de 7 días                              |
| <code>rolling_mean_14</code> | Media móvil de 14 días                             |
| <code>rolling_mean_28</code> | Media móvil de 28 días                             |
| <code>rolling_std_7</code>   | Desviación estándar de 7 días                      |
| <code>rolling_std_14</code>  | Desviación estándar de 14 días                     |
| <code>rolling_std_28</code>  | Desviación estándar de 28 días                     |
| <code>rolling_max_28</code>  | Máximo de ventas en los últimos 28 días            |
| <code>rolling_min_28</code>  | Mínimo de ventas en los últimos 28 días            |

### 3.3.2 Features temporales

**Cuadro 3.3:** Features temporales derivadas del calendario

| Feature                   | Descripción                                                 |
|---------------------------|-------------------------------------------------------------|
| <code>weekday_int</code>  | Día de la semana como entero (1 = sábado, 2 = domingo, ...) |
| <code>day</code>          | Día del mes                                                 |
| <code>week_of_year</code> | Semana del año (1–52)                                       |
| <code>month</code>        | Mes del año (1–12)                                          |
| <code>quarter</code>      | Trimestre del año (1–4)                                     |
| <code>year</code>         | Año                                                         |
| <code>is_weekend</code>   | Indicador binario: 1 si es sábado o domingo                 |
| <code>is_event</code>     | Indicador binario: 1 si hay evento especial ese día         |

### 3.3.3 Features de precio

**Cuadro 3.4:** Features de precio y sus variaciones

| Feature                           | Descripción                                                                                                                  |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>sell_price</code>           | Precio de venta actual del producto en esa tienda                                                                            |
| <code>price_lag_1</code>          | Precio de la semana anterior                                                                                                 |
| <code>price_change</code>         | Diferencia absoluta de precio respecto a la semana anterior                                                                  |
| <code>price_change_pct</code>     | Variación porcentual de precio respecto a la semana anterior                                                                 |
| <code>price_rolling_mean_4</code> | Media del precio en las últimas 4 semanas (28 días)                                                                          |
| <code>price_ratio_vs_avg</code>   | Ratio entre el precio actual y su media de 4 semanas; detecta si el precio está por encima o por debajo de su nivel habitual |

### 3.3.4 Features de producto y tienda (aggregate features)

Las 6 variables categóricas (`item`, `category`, `department`, `store`, `store_code`, `city`) fueron codificadas con **Label Encoding** mediante `sklearn.preprocessing.LabelEncoder`.

**Cuadro 3.5:** Aggregate features de producto y tienda

| Feature                         | Descripción                                                   |
|---------------------------------|---------------------------------------------------------------|
| <code>item_avg_sales</code>     | Media histórica de ventas del producto en todas sus tiendas   |
| <code>store_avg_sales</code>    | Media histórica de ventas de la tienda en todos sus productos |
| <code>category_avg_sales</code> | Media histórica de ventas de la categoría                     |
| <code>city_avg_sales</code>     | Media histórica de ventas de la ciudad                        |

### 3.3.5 Split temporal

Se utilizó un **split temporal estricto** para evitar *data leakage*:

**Cuadro 3.6:** División temporal del dataset

| Conjunto      | Período                 | Filas  | Descripción                              |
|---------------|-------------------------|--------|------------------------------------------|
| Entrenamiento | 2011-01-29 a 2016-02-28 | ~57,4M | 97,2% de los datos                       |
| Test          | 2016-02-29 a 2016-03-27 | ~854K  | 28 días, 1,4%                            |
| Validación    | 2016-03-28 a 2016-04-24 | ~853K  | 28 días más recientes; simula producción |

## 3.4 Clustering

Se realizaron dos análisis de clustering independientes para segmentar productos y tiendas. En ambos casos se aplicó **K-Means** con normalización previa mediante `StandardScaler` y se seleccionó el valor de  $k$  usando el método del codo (*inertia*) y el *silhouette score*.

### 3.4.1 Clustering de productos ( $k = 5$ )

Las *features* utilizadas se calcularon por producto mediante `groupby('item').agg()`.

Evalrados  $k \in \{5, 6, 7\}$ , los *silhouette scores* fueron 0,2249 / 0,1937 / 0,1936. Se seleccionó  $k = 5$  por tener el mejor *score* y la mejor interpretabilidad de negocio.

#### ¿Por qué el Coeficiente de Variación y no la desviación estándar?

El CV se define como  $CV = \sigma / \mu$ . A diferencia de la desviación estándar, normaliza la variabilidad respecto al propio volumen del producto, lo que permite comparar productos de escalas muy distintas. Un producto que vende 100 unidades/día con  $\sigma = 10$  tiene  $CV = 0,10$  (demanda muy estable), mientras que un producto que vende 1 unidad/día con  $\sigma = 2$  tiene  $CV = 2,00$  (demanda muy errática). Si se usara la desviación estándar en bruto, el primero parecería más inestable simplemente porque vende más.

### 3.4.2 Clustering de tiendas ( $k = 3$ )

Evalrados  $k \in \{2, 3, 5\}$ , los *silhouette scores* fueron 0,2660 / 0,2928 / 0,2556. Se seleccionó  $k = 3$  por tener el mejor *score*.

## 3.5 Modelos de predicción

Se evaluaron **5 modelos** comparados contra un *baseline*, todos con el mismo *split* temporal y la misma métrica de evaluación (RMSE sobre el conjunto de test).

### 3.5.1 Justificación de la elección de Gradient Boosting con lag features

Los modelos de ML no tienen noción nativa del tiempo: para ellos cada fila es simplemente un vector de números. Al construir las *lag features* (ventas del día anterior, de hace 7 días, medias móviles, etc.), se le proporciona al modelo la información temporal de forma estructurada, convirtiendo un problema de series temporales en un problema de regresión tabular estándar. Esta es la aproximación dominante en *forecasting* de *retail* a gran escala.

Los resultados del proyecto validan esta elección empíricamente:

Cuadro 3.7: Perfil de los 5 clústeres de productos

| Clúster | Nombre                              | Caracterización                                                                                 | Estrategia de inventario                      |
|---------|-------------------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------|
| C0      | Productos de fin de semana          | HOME & GARDEN dominante. <i>Weekend boost</i> +45%, alta variabilidad ( $cv = 2,79$ )           | Reforzar stock el jueves–viernes              |
| C1      | Productos regulares de supermercado | SUPERMARKET 61%, ventas medias (1,34 u/día), bajan en eventos                                   | Gestión estándar; vigilar días de evento      |
| C2      | Productos de demanda intermitente   | Los que menos venden (0,23 u/día), los más erráticos ( $cv = 3,53$ ), sin patrón claro          | Revisar rentabilidad; posible descatalogación |
| C3      | Productos premium / promocionales   | Precio más alto (14,76€), alta variabilidad de precio ( $\sigma = 0,81$ ), muchas promociones   | Coordinar descuentos con proveedor            |
| C4      | Productos estrella                  | Básicos de SUPERMARKET (93%), las mayores ventas (11,84 u/día), demanda estable ( $cv = 1,18$ ) | Mantener stock siempre; reposición automática |

Cuadro 3.8: Perfil de los 3 clústeres de tiendas

| Clúster | Nombre                    | Caracterización                                                                                                | Estrategia                                               |
|---------|---------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| T0      | Tiendas de baja actividad | Menor volumen (0,92 u/día), menos sensibles al fin de semana (+18%) y a eventos. Mix New York y Philadelphia   | Revisar surtido; posible reducción de stock              |
| T1      | Tiendas de alto volumen   | Mayor volumen (1,26 u/día) pero más erráticas ( $cv = 3,56$ ). Predominantemente Boston                        | Priorizar reposición; gestionar variabilidad             |
| T2      | Tiendas de fin de semana  | Volumen medio (1,07 u/día), mayor efecto fin de semana (+44%), demanda más estable. Predominantemente New York | Reforzar stock jueves–viernes, especialmente en New York |



**Cuadro 3.9:** Comparativa de modelos — rendimiento en test

| Modelo                        | RMSE (test) | Mejora RMSE            |
|-------------------------------|-------------|------------------------|
| Ridge (lineal)                | 2,06        | —                      |
| Random Forest                 | 2,01        | +2,4%                  |
| <b>XGBoost (Grad. Boost.)</b> | <b>1,97</b> | <b>+2,0% adicional</b> |

### 3.5.2 Diferencias entre implementaciones de Gradient Boosting

Los tres modelos de Gradient Boosting comparten el principio de aprendizaje iterativo pero difieren en sus optimizaciones internas:

- **XGBoost:** regularización L1/L2 explícita, crecimiento de árbol por nivel (*level-wise*), mayor robustez ante *overfitting*.
- **LightGBM:** crecimiento por hoja (*leaf-wise*), manejo de datos mediante histogramas, entrenamiento más rápido en *datasets* grandes.
- **CatBoost:** manejo nativo de variables categóricas mediante *ordered target statistics*, reduce la necesidad de *label encoding*.

Para XGBoost y LightGBM se usó *early stopping* (50 *rounds* sin mejora) con `eval_set` sobre el conjunto de test.

## 3.6 Optimización de hiperparámetros (Optuna)

Una vez identificado el mejor modelo, se realizó una **búsqueda bayesiana de hiperparámetros** con la librería **Optuna**, que utiliza el algoritmo *Tree-structured Parzen Estimator* (TPE) para explorar el espacio de hiperparámetros de forma más eficiente que una búsqueda en rejilla (*Grid Search*).

```

1 study = optuna.create_study(direction='minimize') # Minimizar RMSE
2 study.optimize(
3     make_objective(objective_xgb, X_tune, y_tune, X_test, y_test),
4     n_trials=30
5 )

```

**Listing 3.1:** Configuración del estudio Optuna

**Subconjunto de afinación (`X_tune`):** se utilizaron los últimos 5 millones de registros del conjunto de entrenamiento. Esta decisión se tomó por dos motivos: (1) reducir el tiempo de cómputo por *trial* (de ~23 min sobre 57M filas a ~1–2 min sobre 5M filas), y (2) los datos más recientes son más representativos del patrón futuro que los datos de

2011–2013. Los 30 *trials* completaron en  $\sim 46,4$  minutos en total.

**Cuadro 3.10:** Espacio de búsqueda de hiperparámetros XGBoost en Optuna

| Hiperparámetro                | Rango       | Descripción                              |
|-------------------------------|-------------|------------------------------------------|
| <code>n_estimators</code>     | [500, 2000] | Número de árboles                        |
| <code>max_depth</code>        | [4, 8]      | Profundidad máxima del árbol             |
| <code>learning_rate</code>    | [0,01, 0,1] | Tasa de aprendizaje (escala logarítmica) |
| <code>subsample</code>        | [0,6, 1,0]  | Fracción de filas por árbol              |
| <code>colsample_bytree</code> | [0,6, 1,0]  | Fracción de <i>features</i> por árbol    |
| <code>min_child_weight</code> | [1, 50]     | Peso mínimo de hoja                      |
| <code>gamma</code>            | [0,0, 1,0]  | Ganancia mínima para hacer <i>split</i>  |
| <code>reg_alpha</code>        | [0,0, 1,0]  | Regularización L1                        |
| <code>reg_lambda</code>       | [0,5, 2,0]  | Regularización L2                        |

### 3.7 Sistema de abastecimiento

Se desarrolló un sistema de **reposición de inventario semanal** compuesto por dos funciones independientes: `calculate_order()` y `generate_replenishment_orders()`.

#### 3.7.1 Lógica de cálculo de la orden

```

1 def calculate_order(model, current_stock, features_row,
2   safety_stock_days=3):
3     forecast_7d = np.clip(model.predict(features_row), 0, None)
4     expected_demand = forecast_7d.sum()
5     safety_stock = safety_stock_days * forecast_7d.std()
6     reorder_point = expected_demand + safety_stock
7     order_qty = max(0, reorder_point - current_stock)
8     priority = 'ALTA' if current_stock < reorder_point * 0.5
9     else 'NORMAL'
10    ...

```

**Listing 3.2:** Función `calculate_order()`

El **stock de seguridad** se calcula como  $\text{safety\_stock} = \text{safety\_stock\_days} \times \sigma(\text{forecast}_{7d})$ . Esto captura la **incertidumbre intrínseca de la predicción**: si el modelo prevé alta variabilidad en los próximos 7 días, se añade un colchón mayor. Con `safety_stock_days = 3` se cubre, en promedio, 3 días de demanda inesperada.

En un sistema de producción completo, el valor óptimo del stock de seguridad se calcularía estadísticamente a partir del nivel de servicio objetivo:

$$\text{safety\_stock} = Z \cdot \sigma_{\text{demanda}} \cdot \sqrt{\text{lead\_time}} \quad (3.1)$$

donde  $Z$  es el z-score asociado al nivel de servicio deseado (por ejemplo,  $Z = 1,65$  para un nivel de servicio del 95%).

La **prioridad ALTA** se activa cuando el stock actual es menor al 50% del punto de reorden, señalando riesgo inminente de *stockout*.

### 3.7.2 Clasificación por rotación de inventario

Para ilustrar el sistema con una muestra representativa, los productos se clasificaron según su rotación:

```

1 p33 = item_avg_sales.quantile(0.33)
2 p66 = item_avg_sales.quantile(0.66)
3
4 baja_rotacion    = item_avg_sales <= p33
5 media_rotacion   = (item_avg_sales > p33) & (item_avg_sales <= p66)
6 alta_rotacion    = item_avg_sales > p66

```

**Listing 3.3:** Clasificación por rotación de inventario

## 3.8 MLOps — Propuesta de puesta en producción

### 3.8.1 Paso 0 — Refactorización del código

El código del *notebook* debe refactorizarse en módulos Python independientes:

```

src/
+- features/
|   +- build_features.py    <- lógica del feature_engineering_pipeline
+- models/
|   +- predict.py           <- carga best_model.pkl y expone predict(X)
+- api/
|   +- main.py              <- endpoint FastAPI /predict

```

**Listing 3.4:** Estructura de módulos Python para producción

### 3.8.2 Paso 1 — Empaquetado del modelo

**Cuadro 3.11:** Artefactos del modelo generados en el *notebook*

| Artefacto                       | Ruta                       | Contenido                                                |
|---------------------------------|----------------------------|----------------------------------------------------------|
| <code>best_model.pkl</code>     | <code>src/models/</code>   | Modelo XGBoost con parámetros Optuna finales             |
| <code>label_encoders.pkl</code> | <code>src/features/</code> | LabelEncoders ajustados para las 6 variables categóricas |
| <code>feature_list.json</code>  | <code>src/features/</code> | Lista ordenada de <i>features</i> y <i>target</i>        |

### 3.8.3 Paso 2 — Pipeline de predicción semanal

Cada lunes a las 6:00 AM se ejecuta automáticamente el *pipeline* de predicción mediante un orquestador de tareas (Airflow, Prefect o *cron job*):

1. Extraer ventas y precios de la semana anterior (POS/ERP)
2. Calcular features (lags, medias móviles, temporales, precio)
3. Ejecutar XGBoost sobre todas las combinaciones item-store
4. Guardar predicciones en Data Lake
5. Generar `replenishment_orders.csv`
6. Publicar resultados en dashboard Power BI

**Listing 3.5:** Pipeline semanal de predicción

El resultado debe estar disponible antes de las 8:00 AM para que el departamento de compras pueda actuar sobre las órdenes del día.

### 3.8.4 Paso 3 — Monitoreo del modelo en producción

**Cuadro 3.12:** Métricas de monitoreo en producción

| Métrica                         | Frecuencia | Umbral | Justificación                                                                         |
|---------------------------------|------------|--------|---------------------------------------------------------------------------------------|
| MAPE semanal por tienda         | Semanal    | > 67%  | Referencia del modelo: 55,82%. El umbral es el 20% superior ( $55,82\% \times 1,20$ ) |
| RMSE <i>rolling</i> (4 semanas) | Semanal    | > 2,36 | Referencia: RMSE = 1,97. El umbral es el 20% superior ( $1,97 \times 1,20$ )          |

### 3.8.5 Paso 4 — Reentrenamiento

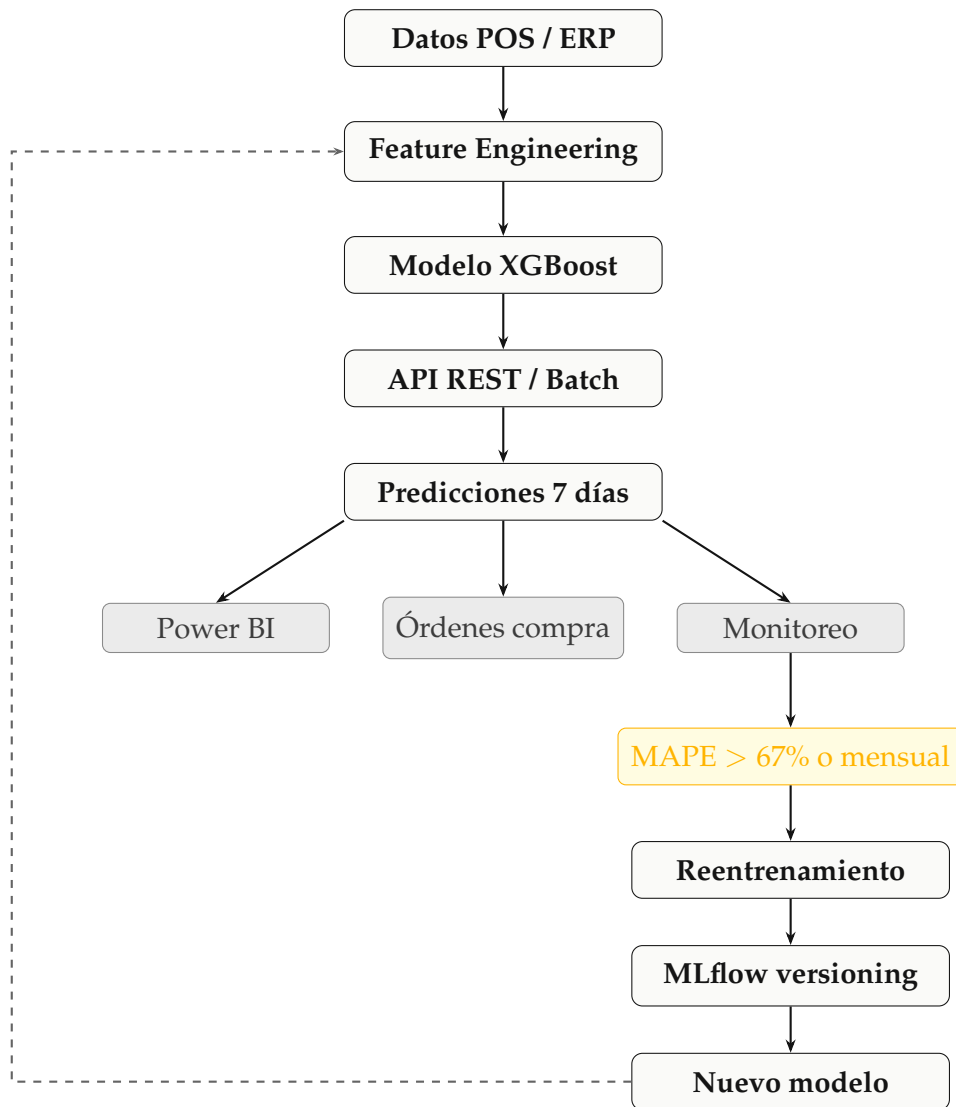
**Cuadro 3.13:** Modalidades de reentrenamiento del modelo

| Modalidad              | Trigger                                   | Estrategia                                                              |
|------------------------|-------------------------------------------|-------------------------------------------------------------------------|
| Programado             | Mensual                                   | Reentrenar con los últimos 2 años de datos + Optuna (30 <i>trials</i> ) |
| Por degradación        | MAPE > 67% durante 2 semanas consecutivas | Reentrenar con los últimos 6 meses                                      |
| Por evento estructural | Nuevo producto / nueva tienda             | Reentrenar <i>on-demand</i> con datos del clúster más similar           |

### 3.8.6 Paso 5 — Versionado y trazabilidad

Se utiliza **MLflow** para registrar cada experimento: parámetros del modelo (hiperparámetros Optuna), métricas (RMSE, MAE, MAPE) en *train*, *test* y validación, y artefactos (`best_model.pkl`, `label_encoders.pkl`, `feature_list.json`).

## 3.8.7 Diagrama del ciclo MLOps



## Resultados y Conclusiones

### 4.1 Resultados del EDA

Los seis hallazgos del análisis exploratorio con implicaciones directas de negocio se resumen en la sección de Metodología (el capítulo anterior). Sus consecuencias operativas más importantes son:

- New York lidera las ventas; las políticas de inventario no deben ser uniformes entre ciudades.
- El fin de semana genera entre un 27–33% más de ventas. Los pedidos deben planificarse para que el stock llegue antes del jueves.
- SUPERMARKET representa el 68,7% de las ventas totales. Un fallo de stock en esta categoría tiene impacto desproporcionado.
- El gran pico de septiembre 2013 fue causado por una bajada agresiva de precios; confirma que las variables de precio son *features* de alto valor predictivo.
- Los eventos especiales tienen impacto asimétrico: SuperBowl (+15,36%), Easter (+12,38%), Thanksgiving (−40,21%).
- La autocorrelación en *lag 1* es de 0,78 (ACF). El test ADF confirma que la serie es estacionaria ( $p$ -valor = 0,0008).

### 4.2 Resultados del Clustering

#### 4.2.1 Clustering de productos ( $k = 5$ )

**Silhouette score: 0,2249.** Los *scores* por debajo de 0,25 son esperables en *retail*: los productos de una misma cadena comparten muchos comportamientos de base (mismos días de la semana, mismos eventos, misma estacionalidad), lo que reduce la separación natural entre clústeres. La elección de  $k=5$  se justifica por tener el mejor *silhouette* y la mejor interpretabilidad de negocio.

#### 4.2.2 Clustering de tiendas ( $k = 3$ )

**Silhouette score: 0,2928.** Los *scores* entre 0,25 y 0,30 indican diferencias comportamentales sutiles entre tiendas, lo cual es lógico al pertenecer todas a la misma cadena y vender el mismo catálogo.

La segmentación en 3 grupos permite:

- Priorizar el presupuesto de reposición en las tiendas de mayor volumen.
- Aplicar estrategias diferenciadas de *safety stock* según la variabilidad histórica del clúster.

### 4.3 Comparativa de modelos predictivos

Todos los modelos fueron evaluados sobre el mismo conjunto de test mediante RMSE.

**Cuadro 4.1:** Resultados comparativos de todos los modelos

| Modelo           | RMSE (test) | T. entrenamiento |
|------------------|-------------|------------------|
| Baseline MA28    | 2,18        | —                |
| Ridge Regression | 2,06        | ~4 min           |
| Random Forest    | 2,01        | ~34 min          |
| CatBoost         | 1,98        | ~56 min          |
| LightGBM         | 1,97        | ~65 min          |
| <b>XGBoost</b>   | <b>1,97</b> | <b>~24 min</b>   |

**Modelo ganador:** XGBoost con RMSE = 1,97, representando una **mejora del 9,6%** sobre el baseline (de 2,18 a 1,97). El criterio definitorio entre XGBoost y LightGBM (ambos con RMSE = 1,97) fue la eficiencia computacional: XGBoost tardó ~24 min frente a los ~65 min de LightGBM, siendo ~3× más rápido con resultados equivalentes.

#### 4.3.1 Nota sobre el MAPE en validación (55,82%)

El MAPE obtenido sobre el conjunto de validación es del **55,82%**, un valor que a primera vista parece elevado pero que es **esperado y normal** en problemas de *forecasting retail* a nivel de granularidad ítem-tienda-día. La fórmula del MAPE,

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \frac{|\hat{y}_i - y_i|}{y_i} \times 100 \quad (4.1)$$

tiene en el denominador el valor real de ventas. Cuando las ventas son bajas (la mayoría de los registros corresponden a productos con 1–5 unidades diarias), dividir entre un número pequeño infla el porcentaje aunque el error absoluto sea mínimo.



**Cuadro 4.2:** Resumen de métricas del modelo final

| Métrica                              | Valor         | Interpretación                                                      |
|--------------------------------------|---------------|---------------------------------------------------------------------|
| <b>RMSE</b>                          | <b>1,97</b>   | El modelo se equivoca $\sim 2$ unidades/día; operativamente válido  |
| Mejora RMSE vs. baseline             | <b>+9,6%</b>  | Supera al baseline; objetivo del proyecto cumplido                  |
| MAPE (validación)                    | 55,82%        | Inflado por baja rotación; no representativo como métrica principal |
| <b>RMSE post-Optuna</b>              | <b>1,8994</b> | Mejora adicional tras optimización bayesiana                        |
| Mejora RMSE post-Optuna vs. baseline | <b>+13,1%</b> | Supera ampliamente el objetivo mínimo del 5%                        |

## 4.4 Importancia de variables (Feature Importance)

**Cuadro 4.3:** Top 20 variables por importancia en el modelo XGBoost optimizado

| Rank | Feature                            | Importance | Interpretación                                 |
|------|------------------------------------|------------|------------------------------------------------|
| 1    | <code>sales_rolling_mean_7</code>  | 1.606.980  | Predictor dominante; $\sim 4\times$ el segundo |
| 2    | <code>sales_rolling_mean_14</code> | 459.264    | Tendencia de 2 semanas                         |
| 3    | <code>sales_rolling_mean_28</code> | 72.202     | Tendencia mensual                              |
| 4    | <code>sales_lag_1</code>           | 62.888     | Ventas de ayer                                 |
| 5    | <code>sales_rolling_min_28</code>  | 60.148     | Suelo de demanda del producto                  |
| 6    | <code>is_weekend</code>            | 34.955     | Efecto fin de semana                           |
| 7    | <code>store_avg_sales</code>       | 27.069     | Volumen base de la tienda                      |
| 8    | <code>weekday_int</code>           | 18.100     | Efecto día-de-semana                           |
| 9    | <code>sales_lag_21</code>          | 17.582     | Memoria de 3 semanas                           |
| 10   | <code>sales_lag_28</code>          | 17.520     | Memoria mensual                                |
| 11   | <code>price_rolling_mean_4w</code> | 14.627     | Precio medio reciente                          |
| 12   | <code>item_encoded</code>          | 12.854     | Identidad del producto                         |
| 13   | <code>quarter</code>               | 12.320     | Estacionalidad trimestral                      |
| 14   | <code>price_lag_1</code>           | 10.440     | Precio semana anterior                         |
| 15   | <code>month</code>                 | 9.225      | Granularidad mensual                           |
| 16   | <code>year</code>                  | 9.071      | Tendencia interanual                           |
| 17   | <code>sales_rolling_std_28</code>  | 9.048      | Volatilidad reciente                           |
| 18   | <code>store_encoded</code>         | 9.038      | Identidad de la tienda                         |
| 19   | <code>price_ratio_vs_avg</code>    | 8.912      | Precio relativo a su media                     |
| 20   | <code>item_avg_sales</code>        | 8.862      | Historial medio del producto                   |

**Interpretación de negocio:** el modelo aprende principalmente de las **medias móviles** y **lags de ventas** (posiciones 1–5, 9–10 y 17 del *ranking*), confirmando que el patrón histórico reciente es el *driver* principal de la demanda en *retail*. El precio aparece en posiciones 11, 14 y 19, confirmando el hallazgo del EDA sobre la elasticidad precio-demanda.

## 4.5 Sistema de reposición de inventario

Cuadro 4.4: Ejemplo de output del sistema de reposición

| item_id        | store_id  | cur. stock | forecast_7d | order_qty | priority |
|----------------|-----------|------------|-------------|-----------|----------|
| SUPERMARKET_2_ | South_End | 5          | 13,5        | ~10       | ALTA     |
| SUPERMARKET_3_ | South_End | 1          | 2,0         | ~1        | ALTA     |
| HOME_&_GARDEN  | South_End | 3          | 4,0         | ~1        | NORMAL   |
| HOME_&_GARDEN  | South_End | 20–100     | 3,0         | 0         | NORMAL   |
| SUPERMARKET_1_ | South_End | 20–100     | 1,2         | 0         | NORMAL   |
| HOME_&_GARDEN  | South_End | 20–100     | 53,7        | 0         | NORMAL   |

## 4.6 Conclusiones finales

### 4.6.1 Conclusiones técnicas

1. El **Gradient Boosting supera claramente a los modelos lineales** en este problema, confirmando la presencia de relaciones no lineales entre *features* y ventas.
2. Las *features de lag* y las **medias móviles son las más informativas**, validando la hipótesis de que los patrones temporales son el *driver* principal de la demanda en *retail*. `sales_rolling_mean_7` domina con un *importance score*  $\sim 4\times$  mayor que el segundo *feature*.
3. **XGBoost ofrece el mejor equilibrio entre rendimiento y eficiencia computacional** para este *dataset* de  $\sim 58\text{M}$  registros, siendo  $\sim 3\times$  más rápido que LightGBM con resultados equivalentes.
4. La **optimización bayesiana (Optuna)** confirmó que el modelo se beneficia de **más árboles y aprendizaje más lento**: los parámetros óptimos encontrados fueron `n_estimators=1933` y `learning_rate=0.023`. `max_depth=8` alcanzó el límite superior del rango de búsqueda, indicando que la complejidad extra está justificada con 57M de filas.
5. El **modelo generaliza bien y no está sobreajustado**: el RMSE en el conjunto de validación (1,8994) es mejor que en el conjunto de test (1,9645).
6. El **split temporal estricto es fundamental** en problemas de series temporales para evitar *data leakage* y obtener estimaciones realistas del rendimiento en producción.

### 4.6.2 Conclusiones de negocio

1. El **sistema reduce la incertidumbre de la gestión de inventario**: el modelo final obtiene un **RMSE = 1,8994** en el conjunto de validación, representando una **mejora del 13,1%** sobre el *baseline*, superando el objetivo mínimo del 5%.

2. El sistema de abastecimiento automatiza una decisión que hoy es manual, liberando tiempo del equipo de operaciones y reduciendo el riesgo de error humano.
3. La segmentación por clustering permite estrategias diferenciadas: no todos los productos y tiendas necesitan el mismo nivel de atención.
4. El proyecto es escalable: la arquitectura desarrollada puede extenderse fácilmente a nuevos productos, tiendas o incluso cadenas del grupo.

## 4.7 Trabajo futuro

Cuadro 4.5: Áreas de mejora propuestas para iteraciones futuras

| Área         | Mejora propuesta                                                                                                                              |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Datos        | Integrar datos climáticos (temperatura, lluvia) y de competencia (precios de competidores, aperturas de tiendas rivales)                      |
| Modelos      | Explorar modelos de series temporales específicos (Prophet, N-BEATS, Temporal Fusion Transformer)                                             |
| Optuna       | Ampliar el rango de búsqueda de <code>max_depth</code> más allá de 8, ya que el óptimo encontrado alcanzó el límite superior del rango actual |
| FE           | Recalcular las <i>aggregate features</i> exclusivamente sobre el conjunto de <i>train</i> para eliminar el <i>data leakage</i> menor presente |
| Producción   | Refactorizar el código del <i>notebook</i> en módulos Python independientes y <i>containerizar</i> con Docker                                 |
| ERP          | Integrar <code>current_stock</code> en tiempo real desde el sistema ERP                                                                       |
| Dashboard    | Extender el dashboard de Power BI con alertas automáticas de <i>stockout</i>                                                                  |
| Safety stock | Calibrar <code>safety_stock_days</code> por clúster de producto en lugar de usar un valor fijo global de 3 días                               |

## Referencias Bibliográficas

(Formato APA 7.<sup>a</sup> edición, organizado por categoría temática.)

### Algoritmos y modelos

- Yandex. (s.f.). *CatBoost documentation*. <https://catboost.ai/docs/>
- Microsoft. (2025). *LightGBM documentation*. <https://lightgbm.readthedocs.io/>
- Optuna Contributors. (2018). *Optuna documentation*. <https://optuna.readthedocs.io/>
- XGBoost developers. (2025). *XGBoost documentation*. <https://xgboost.readthedocs.io/>

### Librerías de Python

- NumPy Development Team. (2008–2026). *NumPy documentation*. <https://numpy.org/doc/>
- pandas Development Team. (2026). *pandas documentation*. <https://pandas.pydata.org/docs/>
- scikit-learn developers. (2007–2026). *scikit-learn documentation*. <https://scikit-learn.org/stable/>

### Dataset y competición de referencia

- Makridakis Competitions. (2020). *M5 Forecasting - Accuracy* [Dataset]. Kaggle. <https://www.kaggle.com/competitions/m5-forecasting-accuracy>
- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022a). The M5 competition: Background, organization, and implementation. *International Journal of Forecasting*, 38(4), 1325–1336. <https://doi.org/10.1016/j.ijforecast.2021.07.007>
- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022b). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4), 1346–1364. <https://doi.org/10.1016/j.ijforecast.2021.10.009>

---

## Metodología

- Dorard, L. (2015). *The Machine Learning Canvas* (v1.0). <http://www.louisdorard.com/machine-learning-canvas>

# ML Canvas — DSMarket

## OWNML Machine Learning Canvas

Designed by: **Grupo 2** | Date: 05/03/2026 | Iteration: 1.2

Version 1.2. Created by Louis Dorard, Ph.D. Licensed under CC BY-SA 4.0. <https://ownml.co>

El *Machine Learning Canvas* es un marco estructurado para describir el ciclo completo de un sistema de ML en producción. Se presentan a continuación los nueve bloques del *canvas* aplicado al proyecto DSMarket.

### 1. Value Proposition

**Beneficiario:** DSMarket (Compras, Operaciones, Dirección).

**Problema actual:** gestión manual del inventario con dos costes críticos:

- *Stockouts*: ventas perdidas y clientes insatisfechos.
- Exceso de stock: capital inmovilizado y productos obsoletos.

**Solución ML:** pasar de una gestión reactiva a una gestión proactiva basada en datos, generando órdenes de reposición automáticas cada semana.

**Valor esperado:** reducción en *stockouts* y exceso de stock. La cuantificación económica requiere datos operativos reales (stock actual, coste de almacenamiento, margen por producto) no disponibles en este proyecto académico.

**Integración:** el *pipeline batch* se ejecuta cada lunes a las 6:00 AM; resultados disponibles antes de las 8:00 AM para el departamento de compras.

## 2. Decisions

**Decisión asistida:** el sistema genera automáticamente órdenes de reposición semanales.

**Parámetros:**

- Horizonte: 7 días.
- Safety stock: `safety_stock_days = 3`.
- Prioridad **ALTA**:  $\text{stock} < 0,5 \times \text{punto reorden}$ .
- Prioridad **NORMAL**: resto de casos.

**Interfaces:** dashboard Power BI + fichero de órdenes exportado al ERP.

**Explicabilidad:** Feature Importance publicada en el dashboard (*top*: `rolling_mean_7`, `rolling_mean_14`, `rolling_mean_28`).

## 3. Prediction Task

**Tipo de tarea:** Aprendizaje supervisado — Regresión.

**Entidad:** combinación ítem-tienda-día.

**Variable objetivo:** `sales` — unidades/día; valor continuo, no negativo, distribución asimétrica positiva.

**Período:** 29 ene 2011 – 24 abr 2016 (1.913 días). Cobertura: 3.049 productos, 10 tiendas, 3 ciudades.

**Modelos:** Baseline (MA28), Ridge, Random Forest, CatBoost, LightGBM, XGBoost.

**Ganador:** XGBoost (RMSE = 1,97 pre-Optuna; 1,899 post-Optuna).

## 4. Impact Simulation

**Métrica principal:** RMSE (en unidades).

**Métrica secundaria:** MAPE.

Baseline (MA28): RMSE = 2,18

XGBoost pre-Optuna: RMSE = 1,97 (+9,6%)

XGBoost post-Optuna: RMSE = 1,899 (+13,1%)

MAPE validación: 55,82%

**Criterio despliegue:**  $\text{RMSE}_{\text{nuevo}} < \text{RMSE}_{\text{anterior}}$   
✓ Cumplido.

**Rollback:** si  $\text{RMSE}_{\text{nuevo}} \geq \text{RMSE}_{\text{anterior}}$ , se revierte automáticamente.

## 5. Data Sources

Tres fuentes internas de DSMarket (2011–2016):

**1. `item_sales.csv`**

Ventas diarias formato *wide*.  
30.490 filas  $\times$  1.913 columnas.

**2. `daily_calendar_with_events.csv`**

Calendario con eventos especiales.

**3. `item_prices.csv`**

Precios semanales ( $\sim 7\text{M}$  registros).

Dataset consolidado (*long*):  $\sim 58,3\text{M}$  registros.

**Origen:** M5 Forecasting (Kaggle, 2020), basado en datos reales de Walmart.



## 6. Data Collection

**Datos históricos:** extraídos de los sistemas internos de DSMarket para el período 2011–2016.

**Actualización continua en producción:**

- Ventas: diariamente desde el sistema POS.
- Precios: semanalmente desde el sistema de *pricing*.
- Ambos flujos mediante ETL automático.

**Etiquetado:** no requiere etiquetado manual; las ventas reales son el *outcome* observado directamente.

**Privacidad:** los datos no contienen información personal (no aplica GDPR); son de uso interno.

**Stock actual:** en el *notebook* se simula con valores aleatorios. En producción debe integrarse desde el ERP en tiempo real.

## 7. Features

20 *features* en 4 categorías:

**Lag** (más importantes):

- `sales_rolling_mean_7, _14, _28`
- `sales_lag_1, _21, _28`
- `sales_rolling_min_28, _std_28`

**Producto/tienda:** `store_avg_sales`,  
`item_encoded`, `store_encoded`,  
`item_avg_sales`.

**Temporales:** `is_weekend`, `weekday_int`,  
`quarter`, `month`, `year`.

**Precio:** `price_rolling_mean_4w`,  
`price_lag_1`, `price_ratio_vs_avg`.

## 8. Building Models

**Modelo en producción:** XGBoost + Optuna (30 *trials*, ~46 min).

**Hiperparámetros óptimos:**

- `n_estimators`: 1.933
- `learning_rate`: 0,023
- `max_depth`: 8
- `min_child_weight`: 31
- `reg_alpha`: 0,342

**Reentrenamiento** (3 modalidades):

- Programado: mensual, últimos 2 años.
- Por degradación: MAPE > 67% durante 2 semanas.
- Por evento estructural: nuevos productos/tiendas.

**Versionado:** MLflow. Artefactos guardados:

`best_model.pkl`, `label_encoders.pkl`,  
`feature_list.json`.

**Split:** Train → Test → Validación.

## 9. Making Predictions

**Modo:** Batch (no *real-time*).

**Frecuencia:** cada lunes a las 6:00 AM.

**Pipeline:** ETL →  
`build_features.py` → `predict.py`  
→ `replenishment_orders.csv` → Power BI.

**Cobertura:** 10 tiendas × 3.049 productos = 30.490 registros.

**Latencia aceptable:** < 2 horas para todo el catálogo.

**Artefactos:** `best_model.pkl`,  
`label_encoders.pkl`,  
`feature_list.json`.

## 10. Monitoring

**Revisión semanal** – indicador implementado:

**MAPE semanal por tienda:** alerta si > 67% (20% de degradación sobre el 55,82% actual). Acción: investigar causa y decidir reentrenamiento.

**RMSE *rolling* (4 semanas):** alerta si > 2,36 (20% sobre RMSE = 1,97).

**Trabajo futuro:** *Stockout rate* real requiere integración con ERP. Una vez disponible, se podría ajustar `safety_stock_days` por clúster de producto.

**Trazabilidad:** todas las versiones del modelo registradas con métricas, hiperparámetros y artefactos.

**Rollback automático:** si

$RMSE_{nuevo} \geq RMSE_{anterior}$ .